Name: Arslan Ahmad Khan.

Class: MAD Section B

Lecturer: Mam Sadaf

Assignment 02

Question 1: Software Requirements Specification (SRS)

1. Introduction

1.1. Purpose

This document defines the complete functional and non-functional requirements for the **Restaurant App MVP**, a frontend-only React Native mobile application designed to streamline customer dining, table reservations, and order management, while providing restaurant managers with a real-time operational dashboard.

1. **Scope**
   In Scope (Added): Node.js & Express REST API backend, MongoDB database persistence (Atlas/Local), JWT-based user authentication, centralized server-side data management for menu items, reservations, and orders.

   Out of Scope (Kept): Real payment gateway integration (Stripe, JazzCash, EasyPaisa API integration), native push notifications, SMS gateways.

   Problems Identified in A1 & Server-side Solutions:

   Problem: **Customer placed an order on one phone, but it didn't appear on another phone.** ***Solution:*** Orders are now saved centrally in MongoDB via POST /api/orders and retrieved dynamically via GET /api/orders/my or GET /api/orders**.**

   Problem: **Manager changes a dish price or availability on one phone, but customers still see old prices/availability.** ***Solution:*** Menu items are fetched directly from MongoDB (GET /api/menu). When a manager updates an item (PUT /api/menu/:id), all clients receive updated data upon screen reload/fetch.

Problem: Reinstalling the app or restarting it clears all order history and table bookings.
Solution: All persistent records are stored in MongoDB collections (orders, reservations, users) instead of local AsyncStorage or mock files.

---

2. Definitions:

Add the following six+ terms to Section 1.3:

1. **API (Application Programming Interface): A set of rules and protocols that allows the mobile app to communicate with the backend server.**
2. **REST (Representational State Transfer): An architectural style for designing networked applications using standard HTTP methods (GET, POST, PUT, PATCH, DELETE).**
3. **Endpoint: A specific URL path on the server where an API can be accessed by the app.**
4. **JSON (JavaScript Object Notation): A lightweight data format used to transmit structured data between the app and the server.**
5. **MongoDB: A document-oriented NoSQL database used to store application data in collections.**
6. **Collection: A grouping of MongoDB documents, equivalent to a table in relational databases.**
7. **Mongoose: An Object Data Modeling (ODM) library for MongoDB and Node.js that manages data relationships and schema validation.**
8. **Middleware: Functions in Express that execute during the request-response cycle to process headers, authenticate tokens, or handle errors.**
9. **JWT (JSON Web Token): A compact, URL-safe means of representing claims to be transferred between two parties securely for user authentication.**
10. **HTTP Status Code: Standardized numerical codes returned by a server indicating the outcome of an HTTP request (e.g., 200, 201, 400, 401, 403, 404, 409, 500).**

2. Overall Description

2.1. User Roles

Role	**Key Permissions & Goals**
Customer	Browse daily specials and full menus, search/filter items, manage cart, apply promo codes, reserve tables, place dine-in/takeaway orders, and track order status in real time.

Manager	Oversee incoming customer orders, update order fulfillment stages manually, accept/decline table reservations, and add/edit menu items, prices, and stock availability.

2.2. User Stories

1. **As a** customer, **I want** to browse daily specials on the home screen, **so that** I can quickly order popular items without scrolling through the entire menu. *(Customer Interview)*
2. **As a** customer, **I want** to filter menu items by categories (Starters, Mains, Desserts, Drinks), **so that** I can easily find what I am craving. *(Customer Interview)*
3. **As a** customer, **I want** to book a table for a specific party size and time slot, **so that** my table is ready upon arrival. *(Customer Interview)*
4. **As a** customer, **I want** to apply promo codes (e.g., WELCOME10), **so that** I can receive discounts on my total bill. *(Customer Interview)*
5. **As a** customer, **I want** to track my order progress (Pending → Preparing → Ready → Served) in real time, **so that** I know when my food will arrive. *(Customer Interview)*
6. **As a** customer, **I want** to toggle between light and dark mode themes, **so that** the app is comfortable to view in low-light environments.
7. **As a** customer, **I want** to search for dishes instantly with debounced inputs, **so that** the UI remains responsive while typing.
8. **As a** customer, **I want** to add custom instructions to my cart items (e.g., "no onions"), **so that** my dietary preferences are communicated accurately.
9. **As a** customer, **I want** to view my past and active reservations, **so that** I can cancel them if my plans change.
10. **As a** manager, **I want** to view all incoming customer orders in a centralized dashboard, **so that** I can eliminate phone-in order errors. *(Manager Goal)*
11. **As a** manager, **I want** to manually update order statuses (Preparing, Ready, Served), **so that** kitchen staff and customers stay synchronized. *(Manager Goal)*
12. **As a** manager, **I want** to toggle menu item availability or update prices instantly, **so that** sold-out items cannot be ordered. *(Manager Goal)*

3. Database Design:

Method	endpoint	access	Request body	respons e
post	/api/auth /register	Public	name, email, password, role	User object & JWT token
Post	/api/auth /login	Public	email, password	User object & JWT token
get	/api/heal th	Public	None	{ status: "ok", time }
Get	/api/men u	Public	None	List of all menu items
Get	/api/men y/:id	Public	None	Single menu item object
post	/api/menu	Manager Only	name, description, price, category, image, isSpecial, isAvailable	Created menu item (201)
put	/api/menu/ :id	Manager Only	Updated menu item fields	Updated menu item object
delete	/api/menu/ :id	Manager Only	None	Success message
Post	/api/orders	Protected	items (menuItemI d, quantity, notes), type, tableId	Created order object (201)
Get	/api/orders /my	Protected	None	List of custome

				r's own orders
Get	/api/orders /:id/status	Manager Only	None	List of all custome r orders
Patch	/api/reserv ations	Protected	Status	Updated order object
Post	/api/reserv ations/my	Protected	tableId, date, timeSlot, partySize	Created reservat ion object (201)
Get	/api/reserv ations	Manager Only	None	List of custome r's reservat ions
Patch	/api/reserv ations/:id	Manager Only	status	Updated reservat ion object

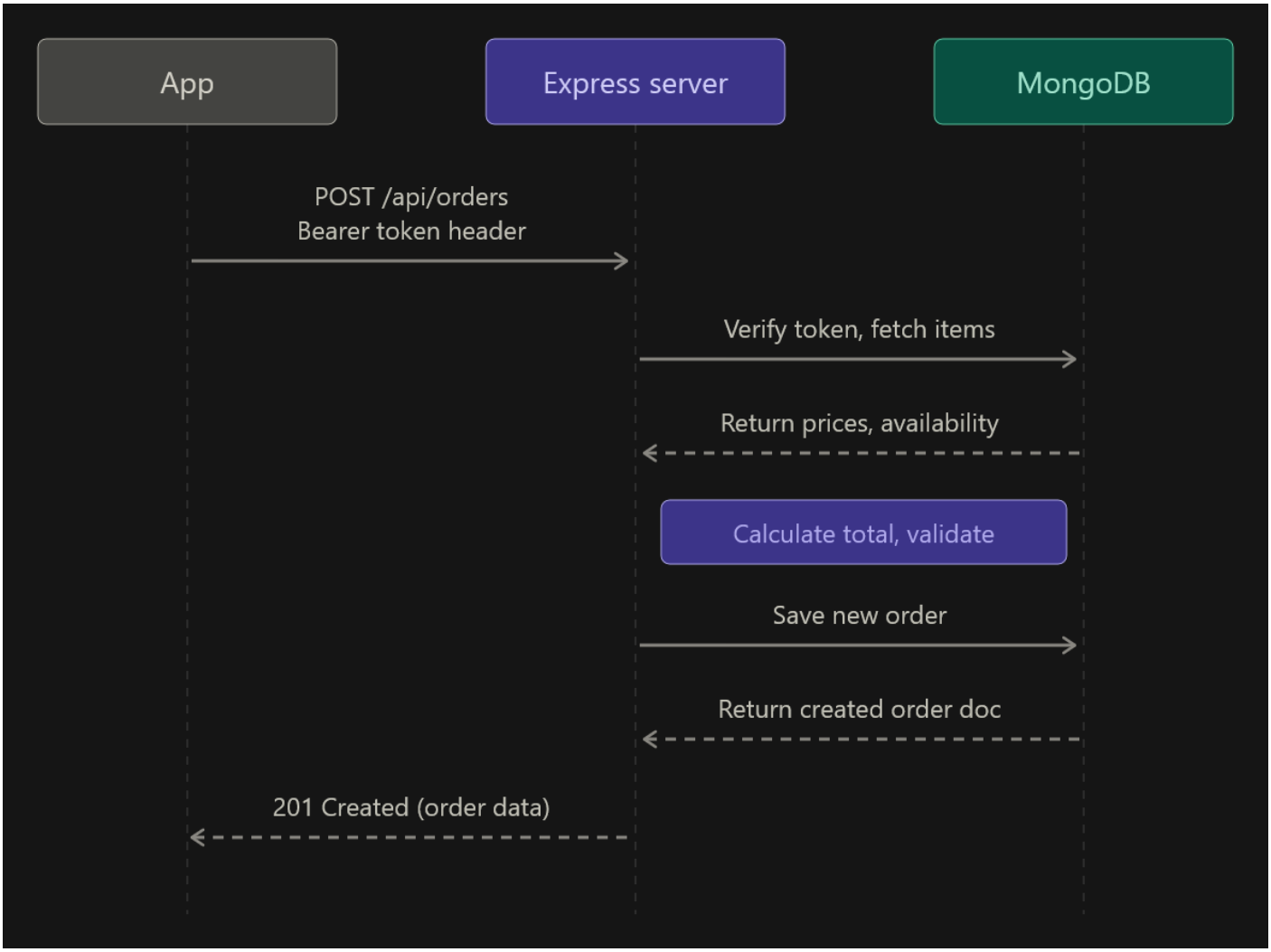
4. Client-Side Data Model (Mock Data)

Data Set Name	Fields	Description
Users	_id (ObjectId) name (String, required) email (String, required, unique) password (String, required, hashed) role (String, enum: ['customer', 'manager'], default: 'customer')	Stores registered accounts. Email must be unique. Password must be bcrypt hashed.

MenuItems	_id (ObjectId) name (String, required) description (String) price (Number, required, min: 0) category (String, required) image (String)	Contains restaurant menu items. Price must not be negative (min: 0).
Reservations	_id (ObjectId) userId (ObjectId, ref: 'User', required) tableId (ObjectId, ref: 'Table', required) date (String, required) timeSlot (String, required) partySize (Number, required, min: 1)	Refers to User (userId) and Table (tableId). Must prevent overlapping bookings for the same table, date, and slot. Timestamps enabled.
Orders	_id (ObjectId) userId (ObjectId, ref: 'User', required) items (Array of { menuItemId: ObjectId, name: String, price: Number, quantity: Number, notes: String }) total (Number, required) type (String, enum: ['Dine-in', 'Takeaway']) status (String, enum: ['Pending', 'Preparing', 'Ready', 'Served', 'Cancelled'], default: 'Pending') tableId (ObjectId, ref: 'Table', optional)	Refers to User (userId) and optionally Table (tableId). Total calculated strictly server-side. Timestamps enabled.

Question 2: UML Diagrams Specifications

Sequence Diagram (Customer Places an Order - 3 Lifelines):



Deployment Diagram:

